

An-Najah National University

Computer Engineering Department

Course: Database Systems

Normalization Document

MixMart Online Store

Online Shopping Store Database Project

Prepared by:

Milad Sam - 12323283

Emad Ataout - 12322222

Submitted to: Dr. Firas Shaka

Date: Spring 2026

Table of Contents

1. Purpose of the Document
2. Database Overview
3. Why Normalization Was Needed
4. First Normal Form (1NF)
5. Second Normal Form (2NF)
6. Third Normal Form (3NF)
7. Short BCNF Note
8. Final Normalized Tables Summary
9. Benefits of Normalization in the Project
10. Conclusion

1. Purpose of the Document

This document explains how normalization was applied in the Mix Mart Online database. The goal is to show that the database was not designed as one large table, but as a group of related tables with clear responsibilities. This helps reduce repeated data, avoid common update problems, and keep the system easier to maintain.

The explanation is written in a student-level style. It focuses on the practical design of the project and connects normalization rules with the actual tables used in the database.

2. Database Overview

The database name used in the project is `online_shopping_store_database`. It supports a Java Swing online shopping system where customers can browse products, add items to cart or wishlist, place orders, write reviews, submit selling requests, and chat with the admin.

Table	Main Purpose	Important Keys / Links
Customer	Stores customer account data	PK: phone_number
Category	Stores product categories	PK: category_id
Product	Stores product details and stock	FK: category_id
ShoppingCart	Stores one active cart for each customer	FK: customer_phone, UNIQUE customer_phone
ShoppingCartItem	Stores products inside carts	FKs: cart_id, product_id
Orders	Stores order header and delivery data	FK: customer_phone
OrderItem	Stores products inside orders	FKs: order_id, product_id
Payment	Stores payment details for orders	FK: order_id, UNIQUE order_id
Shipment	Stores shipping and tracking details	FK: order_id, UNIQUE order_id
Review	Stores product and website reviews	FKs: customer_phone, product_id, order_id
Wishlist	Stores saved products	FKs: customer_phone, product_id

SellingRequest	Stores used products submitted by customers	FKs: customer_phone, category_id
ChatMessage	Stores customer-admin chat messages	FK: customer_phone

3. Why Normalization Was Needed

An online store contains connected information. For example, one customer can place many orders, one order can contain many products, one product belongs to one category, and one customer can write many messages or reviews. If all of this information was stored in one table, the same customer, product, and category data would be repeated many times.

Without normalization, the database could face these problems:

- Update anomaly: changing a product category name would require changing many repeated rows.
- Insertion anomaly: a new category could not be added unless a product already existed for it.
- Deletion anomaly: deleting the last product in a category could accidentally remove category information.
- Data redundancy: customer and product data would be repeated in many orders and carts.

4. First Normal Form (1NF)

A table is in First Normal Form when each field contains a single atomic value and there are no repeating groups. In Mix Mart Online, lists of products are not stored inside one order row or one cart row. Instead, the database uses separate detail tables.

Possible Bad Design	Problem	Normalized Solution
Orders(product1, product2, product3)	Repeating product columns make the table hard to query and limit the number of products.	Orders stores order data, and OrderItem stores each product as a separate row.
ShoppingCart(product_list)	A list inside one field is not atomic and cannot be easily updated.	ShoppingCart stores the cart, and ShoppingCartItem stores each cart product separately.
Review(product_review, website_review in same unclear columns)	The type of review becomes unclear.	Review uses review_type to identify Product or Website review.

Because each row stores one clear fact and each column stores one value, the main tables follow 1NF.

5. Second Normal Form (2NF)

A table is in Second Normal Form when it is already in 1NF and every non-key attribute depends on the whole key. This is especially important in relationship tables such as OrderItem, ShoppingCartItem, and Wishlist.

For example, OrderItem connects an order with a product. It stores quantity and unit_price because these values describe the product inside that specific order. However, product_name and product_description are not repeated in OrderItem because they belong to the Product table.

Table	Correct Attributes Stored	Attributes Not Repeated
OrderItem	order_id, product_id, quantity, unit_price	product_name, category_name, description
ShoppingCartItem	cart_id, product_id, quantity, added_at	customer name, product description
Wishlist	customer_phone, product_id, created_at	customer city, product price

This reduces repeated product and customer data and keeps each table focused on its own purpose.

6. Third Normal Form (3NF)

A table is in Third Normal Form when it is in 2NF and there are no transitive dependencies. In simple words, non-key attributes should depend only on the primary key, not on another non-key attribute.

The project applies this idea by separating independent information into separate tables. For example, category details are stored in Category, not repeated in Product. Payment details and shipment details are stored in their own tables instead of being placed directly inside Orders.

Design Decision	Why It Helps 3NF	Example Table	Related Table
Separate categories from products	Category name depends on category_id, not product_id.	Product	Category
Separate order items from orders	Product details inside an order depend on the order-product relationship.	Orders	OrderItem
Separate payment from orders	Payment status and method describe payment, not the order header.	Orders	Payment
Separate shipment from orders	Tracking number and delivery date describe shipment, not general order data.	Orders	Shipment
Separate chat messages	Messages can grow many times for the same customer.	Customer	ChatMessage

These decisions keep the design cleaner and avoid storing the same facts in multiple places.

7. Short BCNF Note

BCNF is a stronger version of 3NF. A simple way to explain it is that every determinant should be a candidate key. The Mix Mart Online design is suitable for a university project and is close to BCNF in the main practical tables because most dependencies are based on primary keys or unique keys.

Examples: product_id determines product details, category_id determines category details, order_id determines order details, payment_id determines payment details, and message_id determines chat message details. Some tables also use unique constraints, such as one active cart per customer and one payment or shipment record per order.

8. Final Normalized Tables Summary

Table	Normal Form Idea	Main Dependency	Reason
Customer	3NF	phone_number -> Name, City, password	Customer facts are stored once.
Category	3NF	category_id -> category_name, description	Category details are separated from products.
Product	3NF	product_id -> product details	Product data is not repeated in cart/order tables.
ShoppingCart	3NF	cart_id -> customer_phone, status	Cart header is separated from cart items.
ShoppingCartItem	3NF	cart_item_id -> cart, product, quantity	Represents products inside carts.
Orders	3NF	order_id -> order header data	Order header is separated from products, payment, and shipment.
OrderItem	3NF	order_item_id -> order, product, quantity, unit_price	Represents products inside orders.
Payment	3NF	payment_id -> payment details	Payment data is separated from order header.
Shipment	3NF	shipment_id -> shipment details	Shipment data is separated from order header.
Review	3NF	review_id -> rating, comment, type	Reviews are managed separately.
Wishlist	3NF	wishlist_id -> customer, product	Prevents repeated wishlist products.
SellingRequest	3NF	request_id -> submitted item details	Used-product requests are separated from products until approved.
ChatMessage	3NF	message_id -> sender, text, time	Chat history is stored as separate messages.

9. Benefits of Normalization in the Project

Benefit	How it appears in Mix Mart Online
Less repeated data	Customer, product, category, payment, and shipment data are stored in their proper tables.
Better data integrity	Primary keys, foreign keys, unique constraints, and check constraints protect relationships and allowed values.
Easier updates	Changing a category name, product price, or shipment

	status can be done in one clear place.
Cleaner Java code	DAO classes can work with specific tables instead of one large mixed table.
Better reporting	Admin statistics can be calculated more clearly because orders, payments, reviews, and products are organized.

10. Conclusion

The Mix Mart Online database follows the main ideas of normalization in a practical way. The design avoids storing all information in one large table and instead separates customers, products, categories, carts, orders, payments, shipments, reviews, wishlist items, selling requests, and chat messages into related tables.

The database design mainly satisfies 1NF, 2NF, and 3NF. It also uses primary keys, foreign keys, unique constraints, check constraints, and default values to protect the data. Overall, the normalized structure makes the system easier to maintain, easier to explain, and more suitable for a real database-backed shopping application.